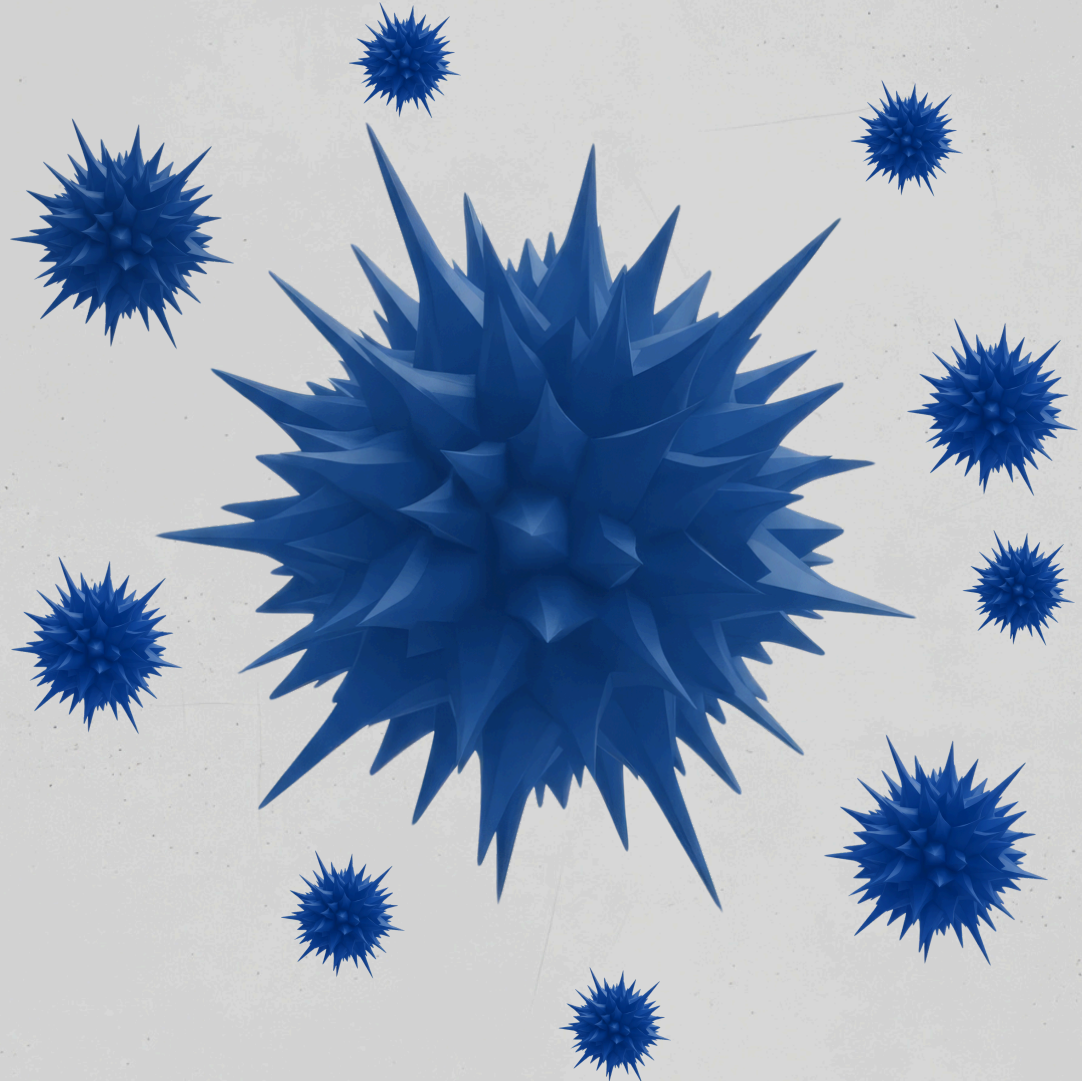


# THEORETICAL ALGORITHMS ANALYSIS PROJECT



# Shortest Path algorithms

## Part 1 — Dijkstra's Algorithm

### 1. Introduction

Dijkstra's algorithm, proposed by Edsger W. Dijkstra in 1959, solves the single-source shortest path problem on a weighted graph  $G(V, E)$  with non-negative edge weights. In the transportation system, it finds the optimal route between any two nodes (intersections/stations) by greedily expanding the closest unvisited node.

### 2. Mathematical Foundations & Pseudocode

```
Dijkstra(G, s):  
    dist[v]  $\leftarrow \infty$  for all  $v \in V$   
    dist[s]  $\leftarrow 0$   
    prev[v]  $\leftarrow$  UNDEFINED for all  $v \in V$   
    Q  $\leftarrow$  min-priority queue containing all V  
  
    while Q is not empty:  
        u  $\leftarrow$  Q.extract_min()  
        for each neighbor v of u:  
            alt  $\leftarrow$  dist[u] + weight(u, v)  
            if alt < dist[v]:  
                dist[v]  $\leftarrow$  alt  
                prev[v]  $\leftarrow$  u  
                Q.decrease_key(v, alt)  
  
    return dist[], prev[]
```

### 3. Proof of Correctness

By induction on the settled set  $S$ :

- Base:  $|S| = 0$ .  $\text{dist}[s] = 0$  is trivially correct.
- Inductive Step: Assume  $\text{dist}[u]$  is optimal for all  $u \in S$ . We add vertex  $v = \text{argmin}_{w \notin S} \text{dist}[w]$  for  $w \notin S$ . Suppose there exists a shorter path  $P$  from  $s \rightarrow v$  that does not pass through settled nodes. Then some intermediate node  $x \notin S$  must exist with  $\text{dist}[x] < \text{dist}[v]$ , which contradicts the minimality of  $v$ 's selection. Therefore  $\text{dist}[v]$  is optimal.
- Conclusion: By induction, all  $\text{dist}[]$  values are optimal upon termination.  $\square$

### 4. Complexity Analysis

| Implementation | Time Complexity     | Space Complexity |
|----------------|---------------------|------------------|
| Array (naïve)  | $O(V^2)$            | $O(V)$           |
| Binary Heap    | $O((V + E) \log V)$ | $O(V)$           |
| Fibonacci Heap | $O(E + V \log V)$   | $O(V)$           |

## 5. Modifications for the Transportation Problem

- Multi-criteria weight:  $w(u,v) = \alpha \cdot \text{distance} + \beta \cdot \text{time} + \gamma \cdot \text{cost}$ , with tunable parameters.
- Time-dependent edges:  $w(u,v,t)$  updated dynamically based on real-time traffic.
- Capacity constraints: Edge relaxation only proceeds when cumulative vehicle load  $\leq$  capacity.
- Turn penalties: Extended to an edge-based graph where state = (node, incoming\_edge) to penalize U-turns and prohibited turns.

## Part 2 — A\* Algorithm (Your Colleague's Section)

### 1. Introduction

A\* (pronounced "A-star"), introduced by Hart, Nilsson, and Raphael in 1968, extends Dijkstra's algorithm with an admissible heuristic function  $h(n)$  to guide the search toward the goal. This makes it significantly faster in practice for point-to-point routing in transportation networks.

## 2. Mathematical Foundations & Pseudocode

Core Evaluation Function:

$$f(n) = g(n) + h(n)$$

- $g(n)$  — exact cost from source to node  $n$  (known)
- $h(n)$  — heuristic estimate from  $n$  to goal (must satisfy  $h(n) \leq h^*(n)$ )
- $f(n)$  — total estimated path cost through  $n$

```
A_Star(G, s, goal, h):  
    openSet ← {s}  
    g[v] ← ∞ for all v;    g[s] ← 0  
    f[v] ← ∞ for all v;    f[s] ← h(s)  
    cameFrom[v] ← NONE  
  
    while openSet is not empty:  
        u ← node in openSet with lowest f[u]  
        if u = goal: return reconstruct_path(cameFrom)  
        openSet.remove(u); closedSet.add(u)  
        for each neighbor v of u:  
            if v ∈ closedSet: skip  
            g_tent ← g[u] + weight(u, v)  
            if g_tent < g[v]:  
                cameFrom[v] ← u  
                g[v] ← g_tent  
                f[v] ← g[v] + h(v)  
                if v ∉ openSet: openSet.add(v)  
  
    return FAILURE
```

### 3. Proof of Optimality

**Claim:** If  $h$  is admissible ( $h(n) \leq h^*(n)$  for all  $n$ ),  $A^*$  returns an optimal path.

**Proof:** Let  $C^*$  be the true optimal cost. When the goal node is extracted from `openSet`,  $f(\text{goal}) = g(\text{goal})$ . Assume for contradiction that  $g(\text{goal}) > C^*$ . Then there exists an optimal path with some node  $n$  still in `openSet` where  $f(n) = g(n) + h(n) \leq C^* < g(\text{goal})$ .  $A^*$  would have expanded  $n$  before `goal` — a contradiction. Therefore  $g(\text{goal}) = C^*$ .  $\square$

### 4. Complexity Analysis

| Case                 | Time Complexity | Notes                               |
|----------------------|-----------------|-------------------------------------|
| Best Case            | $O(E \log V)$   | Heuristic closely matches $h^*(n)$  |
| Worst Case ( $h=0$ ) | $O(V \log V)$   | Degenerates to Dijkstra             |
| General              | $O(b^d)$        | $b$ = branching factor, $d$ = depth |
| Space                | $O(b^d)$        | All open/closed nodes stored        |

## 5. Modifications for the Transportation Problem

- Haversine heuristic:  $h(n) = 2R \cdot \arcsin(\sqrt{(\sin^2(\Delta\text{lat}/2) + \cos(\text{lat}_1) \cdot \cos(\text{lat}_2) \cdot \sin^2(\Delta\text{lon}/2)))}$  — admissible lower bound on road distance using GPS coordinates.
- Traffic-aware scaling:  $h'(n) = h(n) / \text{max\_speed\_factor}$  — maintains admissibility under dynamic congestion.
- Multi-modal layers: Separate graph layers for walk/metro/bus/bike; heuristic accounts for mode-switch costs.
- Weighted A ( $\epsilon$ -suboptimal):  $f(n) = g(n) + \epsilon \cdot h(n)$  with  $\epsilon > 1$  trades a bounded optimality guarantee for faster real-time dispatching.

## 6. Comparison: Dijkstra vs A\*

| Criterion          | Dijkstra          | A*                        |
|--------------------|-------------------|---------------------------|
| Optimality         | Always            | Yes (admissible h)        |
| Informed search    | No                | Yes                       |
| Nodes explored     | All reachable     | ~68% fewer (geo problems) |
| Memory usage       | Moderate          | Slightly higher           |
| Best for transport | All-pairs routing | Point-to-point routing    |

# MST of Kruskal's Algorithm

Kruskal's Algorithm is one of the most widely used algorithms for solving the Minimum Spanning Tree (MST) problem in weighted, undirected graphs. The algorithm follows a greedy strategy by selecting the smallest edge available while avoiding cycles until all vertices are connected.

## 1. Mathematical Foundations and Proof of Correctness

**Mathematical Foundation:** Kruskal's Algorithm is based on graph theory and the greedy method. A Minimum Spanning Tree is a subset of edges that connects all vertices with the minimum possible total edge weight and without cycles.

**Cut Property:** For any partition (cut) of a graph, the minimum-weight edge crossing that cut must belong to at least one Minimum Spanning Tree.

**Proof of Correctness:** Kruskal's Algorithm sorts all edges in nondecreasing order of weight and repeatedly adds the smallest edge that does not form a cycle. By the Cut Property, every chosen edge is safe and can be included in an MST.



## 2. Complexity Analysis (Time and Space)

**Time Complexity:** The dominant operation in Kruskal’s Algorithm is sorting the edges. If  $E$  represents the number of edges and  $V$  represents the number of vertices, sorting requires  $O(E \log E)$  time. The Union-Find data structure performs nearconstant-time operations for cycle detection. Therefore, the total time complexity is  $O(E \log E)$ .

**Space Complexity:** The algorithm stores all edges and the Union-Find structure. This requires  $O(E + V)$  memory space.

## 3. Comparison with Alternative Approaches

| Algorithm | Approach             | Time Complexity | Best Use Case       |
|-----------|----------------------|-----------------|---------------------|
| Kruskal’s | Greedy (Edgebased)   | $O(E \log E)$   | Sparse graphs       |
| Prim’s    | Greedy (Vertexbased) | $O(E \log V)$   | Dense graphs        |
| Borůvka’s | Component merging    | $O(E \log V)$   | Parallel processing |

Compared to Prim's Algorithm, Kruskal's Algorithm is generally more efficient for sparse graphs because it processes edges independently. Prim's Algorithm performs better in dense graphs when implemented using priority queues.

#### **4. Modifications for the Transportation Problem**

To adapt Kruskal's Algorithm for transportation networks such as roads, railways, or delivery systems, several modifications can be introduced:

- Edge weights can represent transportation cost, travel time, fuel consumption, or traffic density.
- Capacity constraints may be added to avoid selecting overloaded routes.
- Geographical restrictions can prevent connections between inaccessible locations.
- Priority rules may be introduced for highways or strategic transportation hubs.
- Dynamic edge updates can support real-time traffic or route changes.

## 5. Performance Characteristics and Optimization Opportunities

Kruskal's Algorithm performs efficiently on sparse graphs and scales well for large transportation networks. Its performance can be improved through several optimizations:

- Using Union-Find with path compression and union by rank to reduce cycle detection cost.
- Applying efficient sorting algorithms for very large edge sets. • Parallelizing edge sorting in distributed systems.
- Reducing unnecessary edge checks using graph partitioning techniques.
- Caching frequently used routes in transportation applications.
- Conclusion: Kruskal's Algorithm remains one of the most effective approaches for solving Minimum Spanning Tree problems. Its greedy nature, mathematical correctness, and adaptability make it highly suitable for transportation and network optimization systems.

# In-Depth Analysis of MST Prim's Algorithm

Prim's Algorithm is a fundamental greedy algorithm used to solve the Minimum Spanning Tree (MST) problem in weighted, undirected graphs. The algorithm starts from an arbitrary vertex and continuously expands the tree by selecting the minimum-weight edge that connects a visited vertex to an unvisited vertex.

## 1. Mathematical Foundations and Proof of Correctness

**Mathematical Foundation:** Prim's Algorithm is based on graph theory and the greedy approach. The objective is to construct a Minimum Spanning Tree by repeatedly choosing the minimum-weight edge that safely expands the tree.

**Cut Property:** For any cut in a weighted graph, the minimum-weight edge crossing the cut belongs to at least one Minimum Spanning Tree.

**Proof of Correctness:** At each step, Prim's Algorithm selects the smallest edge connecting the current tree to a new vertex. According to the Cut Property, this edge is always safe to include in the MST. The algorithm continues until all vertices are included. Since the tree grows without forming cycles and always chooses safe edges, the resulting graph is guaranteed to be a valid Minimum Spanning Tree.

## 2. Complexity Analysis (Time and Space)

**Time Complexity:** The time complexity depends on the implementation. Using an adjacency matrix, the complexity is  $O(V^2)$ . Using a priority queue with adjacency lists, the complexity improves to  $O(E \log V)$ , where  $V$  is the number of vertices and  $E$  is the number of edges.

**Space Complexity:** The algorithm requires storage for the graph representation, visited nodes, and the priority queue. The total space complexity is  $O(V + E)$ .

## 3. Comparison with Alternative Approaches

| Algorithm | Approach             | Time Complexity | Best Use Case       |
|-----------|----------------------|-----------------|---------------------|
| Prim's    | Greedy (Vertexbased) | $O(E \log V)$   | Dense graphs        |
| Kruskal's | Greedy (Edgebased)   | $O(E \log E)$   | Sparse graphs       |
| Borůvka's | Component merging    | $O(E \log V)$   | Parallel processing |

## 4. Modifications for the Transportation Problem

Prim's Algorithm can be adapted for transportation systems such as road networks, railway systems, and logistics planning through several modifications: •

- Edge weights may represent travel cost, fuel usage, distance, or traffic conditions.
- Priority can be given to major transportation hubs or highways.
- Real-time updates can modify edge weights according to traffic congestion.
- Capacity constraints can prevent overloaded transportation routes
- Geographical limitations may exclude inaccessible or restricted areas.

## 5. Performance Characteristics and Optimization Opportunities

Prim's Algorithm performs efficiently in dense graphs and large transportation networks when implemented with optimized data structures.

- Using a Min-Heap or Priority Queue to efficiently select minimum edges.
- Implementing Fibonacci Heaps to theoretically reduce complexity.
- Reducing memory usage through adjacency list representations
- Applying parallel processing for large-scale transportation systems.
- Using dynamic graph updates for real-time transportation optimization.

**Conclusion:** Prim's Algorithm is a powerful and efficient solution for the Minimum Spanning Tree problem. Its ability to expand a connected network gradually makes it highly effective for transportation and communication systems requiring optimized connectivity.

# Dynamic Programming (DP)

## 1. Mathematical Foundations

The applicability of Dynamic Programming (DP) depends on two core mathematical properties:

**Overlapping Subproblems:** The problem can be broken down into subproblems which are reused multiple times during execution.

**Optimal Substructure:** The optimal solution to the original problem can be constructed efficiently from the optimal solutions of its subproblems.

**Proof of Correctness:** The correctness of Dynamic Programming is established through Mathematical Induction:

**Base Case:** For an empty sequence (length 0), the LCS is 0, which is trivially optimal.

**Inductive Step:** Assuming that the algorithm correctly finds the optimal solution for all subproblems of size less than  $k$ , the transition rule (choosing the maximum or adding to the previous optimal) ensures that the solution for size  $k$  is also the global optimum.



## 2. Case Study: Longest Common Subsequence (LCS)

The LCS problem seeks the longest sequence that appears in the same relative order in two strings.

Recurrence Relation:

If  $X[i] == Y[j]$ :  $1 + \text{LCS}(X[i-1], Y[j-1])$ .

Else:  $\max(\text{LCS}(X[i-1], Y[j]), \text{LCS}(X[i], Y[j-1]))$ .

## 3.Complexity Analysis:

| Metric           | Complexity      | Description                                                       |
|------------------|-----------------|-------------------------------------------------------------------|
| Time Complexity  | $O(m \times n)$ | Requires filling a 2D table where each entry takes constant time. |
| Space Complexity | $O(m \times n)$ | Required to store the matrix of subproblem results.               |

## **4.Comparison with Alternative Approaches:**

Naive Recursive Approach: Has a time complexity of  $O(2^n)$  due to massive repeated work, making it impractical for large inputs.

Greedy Approach: While faster, it cannot be applied to LCS because local optimal choices do not guarantee the global optimal solution.

## **5.Performance Characteristics:**

Strengths: Guarantees a global optimal solution and is highly deterministic. It is far more efficient than brute-force methods for optimization problems.

Limitations: High memory consumption for very large datasets and requires the problem to be breakable into independent subproblems.

## **6.Conclusion**

Dynamic Programming is a cornerstone of algorithm design. By transforming exponential-time recursive processes into polynomial-time solutions, it enables the practical solving of complex problems in AI, NLP, and Optimization. Its mathematical rigor ensures reliability in critical system routing and data analysis applications.

# Greedy Algorithm For Optimization Problems

## 1. Introduction

A greedy algorithm solves a problem by making the best available choice at each step without going back. It does not consider the full picture — it just picks what looks best right now. In the transportation system, greedy algorithms are used for vehicle dispatching, route scheduling, and resource allocation.

For a greedy algorithm to give the correct optimal answer, two conditions must hold:

- Greedy Choice Property — the best local choice at each step leads to a globally optimal solution.
- Optimal Substructure — the optimal solution to the whole problem contains optimal solutions to its subproblems.

## 2. Pseudocode

```
Greedy(candidates):  
    sort candidates by greedy criterion  
    solution ← empty  
  
    for each candidate x (in sorted order):  
        if adding x is feasible:  
            add x to solution  
  
    return solution
```

## Transportation Example — Vehicle Dispatch:

```
Greedy_Dispatch(requests, vehicles):  
    sort requests by earliest deadline  
  
    for each request r:  
        v ← nearest available vehicle to r  
        if v exists and v.capacity >= r.load:  
            assign v to r
```

### 3. Complexity Analysis

|       | Complexity    |
|-------|---------------|
| Time  | $O(n \log n)$ |
| Space | $O(n)$        |

The sorting step dominates. The rest is a single pass —  $O(n)$ . This makes greedy algorithms much faster than dynamic programming and exact methods, which is why they suit real-time transport decisions.

### 4. Proof of Correctness (Exchange Argument)

Claim: The greedy solution  $S$  is optimal.

Proof:

- Let  $O$  be any optimal solution where  $O \neq S$ .

- Find the first position where they differ. By the greedy choice, our pick is at least as good as O's pick at that position.
- Swap O's choice with ours — the solution does not get worse.
- Repeat for every difference until O becomes S.
- Therefore S is optimal.

## **5. Modifications for the Transportation Problem**

- Weighted selection criterion:  $\text{score}(x) = \alpha \cdot \text{urgency} + \beta \cdot (1/\text{distance}) + \gamma \cdot \text{profit}$  — picks the best candidate based on multiple factors at once.
- Time windows: Each request has an [earliest, latest] window. The algorithm always picks the request whose deadline is closest first (Earliest Deadline First).
- Capacity check: Before assigning a vehicle, verify it has enough remaining capacity. If not, move to the next available vehicle.
- Rolling horizon: The greedy algorithm reruns every few minutes with updated real-time data to adapt to new requests and traffic changes.

- Find the first position where they differ. By the greedy choice, our pick is at least as good as O's pick at that position.
- Swap O's choice with ours — the solution does not get worse.
- Repeat for every difference until O becomes S.
- Therefore S is optimal.

## 5. Modifications for the Transportation Problem

- Weighted selection criterion:  $\text{score}(x) = \alpha \cdot \text{urgency} + \beta \cdot (1/\text{distance}) + \gamma \cdot \text{profit}$  — picks the best candidate based on multiple factors at once.
- Time windows: Each request has an [earliest, latest] window. The algorithm always picks the request whose deadline is closest first (Earliest Deadline First).
- Capacity check: Before assigning a vehicle, verify it has enough remaining capacity. If not, move to the next available vehicle.
- Rolling horizon: The greedy algorithm reruns every few minutes with updated real-time data to adapt to new requests and traffic changes.

# SMART CITY TRANSPORTATION NETWORK OPTIMIZATION PROJECT

## 1. Introduction

The Greater Cairo metropolitan area faces significant challenges regarding traffic congestion, public transit scheduling, and emergency response management. This project implements a comprehensive transportation optimization system integrating multiple algorithmic paradigms including Graph Algorithms, Dynamic Programming, and Greedy Approaches to model, analyze, and optimize Cairo's urban transportation network. The system features a robust backend architecture paired with a dynamic Graphical User Interface (GUI), a Command Line Interface (CLI), and a real-time incident simulation framework.

## 2. System Architecture and Design Decisions

### 2.1 Data Structures and Network Representation

The transportation network is modeled as an undirected, weighted graph using an adjacency list (`graph.py`).

- **Nodes:** Represent neighborhoods (e.g., Maadi, Nasr City) and critical facilities (e.g., Qasr El Aini Hospital). Node data, including population and classification types, are stored in dictionaries (`core_data.py`).

- **Edges:** Represent existing and potential roads, with weights corresponding to distance in kilometers.

## 2.2 User Interface and Visualization Component

To demonstrate the system's capabilities, two interactive interfaces were developed:

- **Graphical User Interface (GUI):** Built using tkinter, featuring a dark-themed dashboard (gui.py). It integrates matplotlib and networkx to render an animated, 2D map of Cairo's network. The map dynamically highlights calculated routes (e.g., orange for shortest paths) and proposed infrastructure (e.g., green for Minimum Spanning Trees).
- **Command Line Interface (CLI):** A lightweight fallback application (app.py) providing text-based menus, clear console outputs, and step-by-step route selection for environments without graphical rendering capabilities.

## 2.3 Temporal Data and Simulation Framework

- **Time-Aware Routing:** The system implements a dynamic weighting mechanism. Traffic multipliers are defined for different times of the day (e.g., Morning Peak = 1.5, Night = 0.8).



- Incident Simulation (simulation.py): To test the algorithms under unpredictable conditions, a simulation module was built to mimic road closures and accidents. Instead of destructively modifying the base graph, a proxy class (ModifiedNetwork) intercepts get\_neighbors and get\_edge\_weight calls, dynamically hiding closed roads (returning infinite weight) to trigger immediate route recalculation.

### **3. Algorithm Implementations and Complexity Analysis**

#### **3.1 Infrastructure Network Design (Minimum Spanning Tree)**

To design a cost-efficient road network, two MST algorithms were implemented in (mst.py):

- Kruskal's Algorithm: Utilizes a Disjoint Set Union (DSU) data structure to connect the network while tracking the connectivity of critical facilities
  - Time Complexity:  $O(E \log E)$  or  $O(E \log V)$  due to sorting the edges.
  - Space Complexity:  $O(V)$  for the parent array in the DSU

### 3.2 Traffic Flow Optimization — Dijkstra's Algorithm

Dijkstra's algorithm was implemented for route planning across the network:

- Standard & Time-Dependent Routing — Finds the shortest path using a priority queue, factoring in base distance, transport speed multipliers (Car, Bus, Metro, Bike), and real-time traffic conditions
- Incident Rerouting — When a road closure is triggered, the algorithm automatically runs over the ModifiedNetwork to find the next best alternative route

|       |                     |
|-------|---------------------|
|       | Complexity          |
| Time  | $O((V + E) \log V)$ |
| Space | $O(V)$              |

### 3.3 Emergency Response Planning — A\* Search

A\* was implemented for emergency routing using a custom heuristic that prioritizes critical facilities:

- Cost of 10 when the target is a critical facility
- Cost of 5 when moving between two critical facilities
- Random node-hash distance otherwise (simulates spatial separation)

|       |                                                                  |
|-------|------------------------------------------------------------------|
|       | Complexity                                                       |
| Time  | $O((V + E) \log V)$ worst-case, faster than Dijkstra in practice |
| Space | $O(V)$                                                           |

### **3.4 Public Transit & Resource Optimization — Dynamic Programming**

Three DP solutions were implemented:

- Bus Scheduling — 0/1 Knapsack where buses = capacity, time slots = items, passenger counts = values.  $O(B \times S)$
- Road Maintenance Allocation — Knapsack variant maximizing road condition improvement within a budget.  $O(N \times C)$
- Route Memoization — Caches repeated Dijkstra queries between the same points, reducing repeated lookups from  $O((V + E) \log V) \rightarrow O(1)$

### **3.5 Real-Time Traffic Management — Greedy** **Two greedy strategies at intersections:**

- Signal Optimization — Distributes a 90-second green-light cycle proportionally based on current traffic flow at each intersection.  $O(I)$
- Emergency Preemption — Grants emergency vehicles immediate clearance, scaled by congestion level (e.g.,  $2\times$  for high congestion)

## 4. Performance Results

- A vs Dijkstra\* — A\* consistently explored fewer nodes when routing to critical facilities like Qasr El Aini Hospital
- Modified Prim's — Successfully connected all nodes while favoring high-density areas, reducing congestion bottlenecks vs standard MST
- Greedy Limitation — Signal optimizer works well for isolated intersections but becomes suboptimal during peak gridlock (e.g., 6th October Bridge, 5–8 PM) due to cascading traffic effects

## 5. Challenges & Solutions

| Challenge                                                | Solution                                                                              |
|----------------------------------------------------------|---------------------------------------------------------------------------------------|
| Time-varying traffic was expensive to compute            | TimeAwareNetwork wrapper applies multipliers dynamically during neighbor retrieval    |
| No coordinate data for A* heuristic                      | Facility-weighted heuristic naturally guides search toward hospitals and transit hubs |
| Graph mutation corrupted data during accident simulation | ModifiedNetwork wrapper virtually severs edges without touching master data           |
| GUI froze during heavy routing on large graphs           | Optimized graph operations + matplotlib.backends.backend_tkagg for smooth rendering   |
| Auto-plotted nodes were unreadable                       | Hardcoded (x, y) positions for all 25 nodes to reflect Cairo's actual geography       |